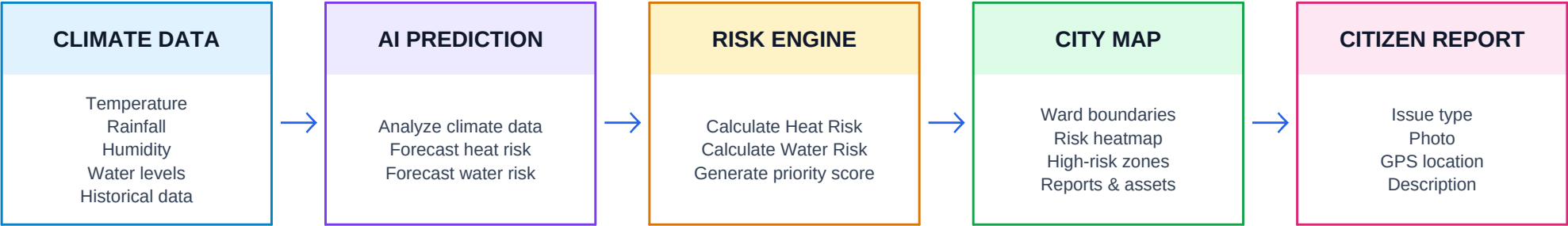


CLIMATE SMART CITY

Complete Application User Flow

1. SYSTEM OVERVIEW



The system converts environmental data into location-specific risk, then connects that risk with real-world citizen reports and authority action.

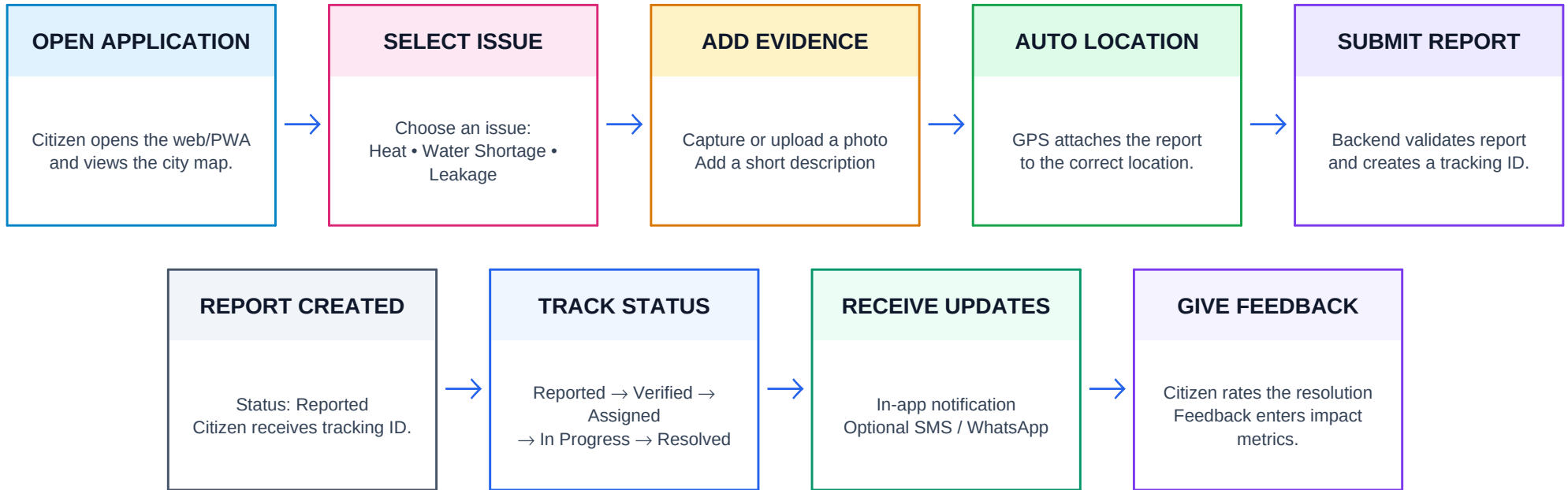


CORE FLOW Climate Data → AI Prediction → Risk → Map → Citizen Report → Authority Action → Impact Tracking → Feedback

CLIMATE SMART CITY

Citizen Application Flow

2. CITIZEN JOURNEY

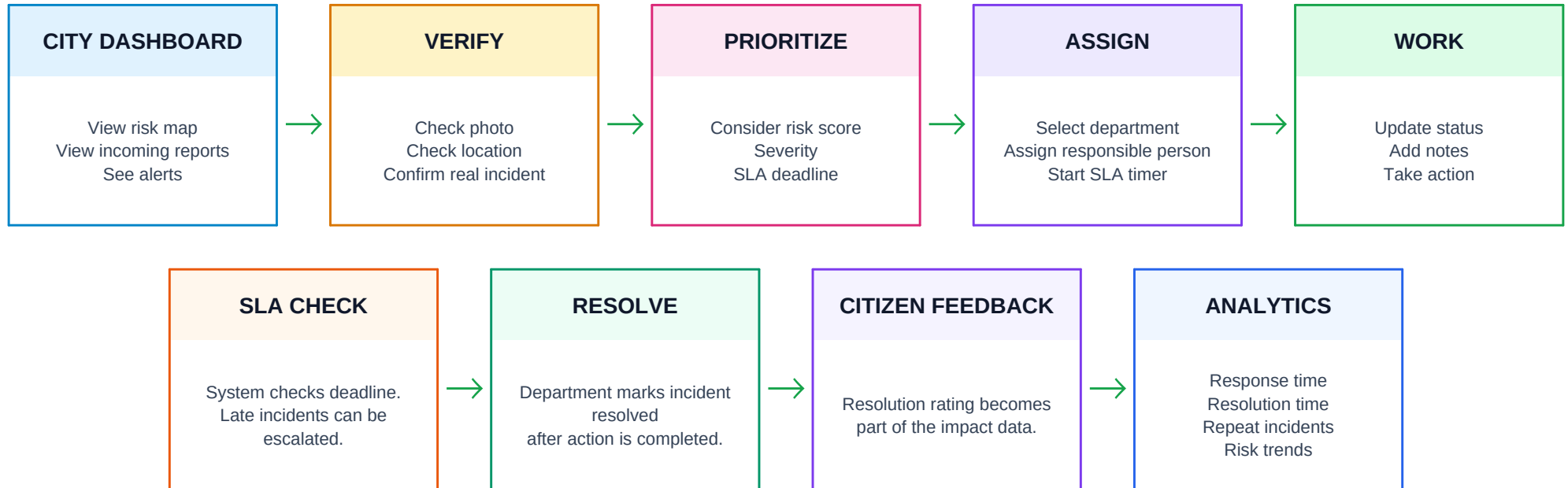


Important: The citizen does not need to understand the AI model. The citizen only reports the real-world problem; the platform handles location, risk context and workflow.

CLIMATE SMART CITY

Authority / Department Flow

3. OPERATIONS JOURNEY

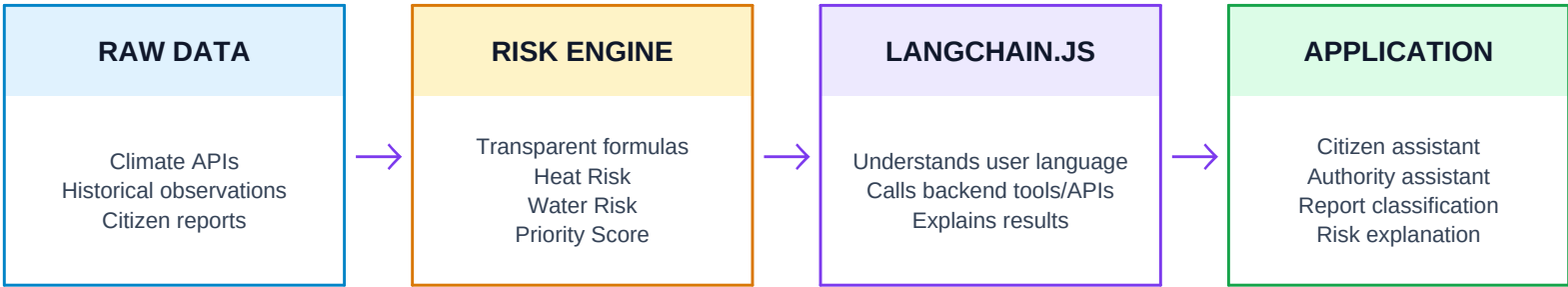


Authority goal: move an incident from Reported → Verified → Assigned → In Progress → Resolved while maintaining an auditable history.

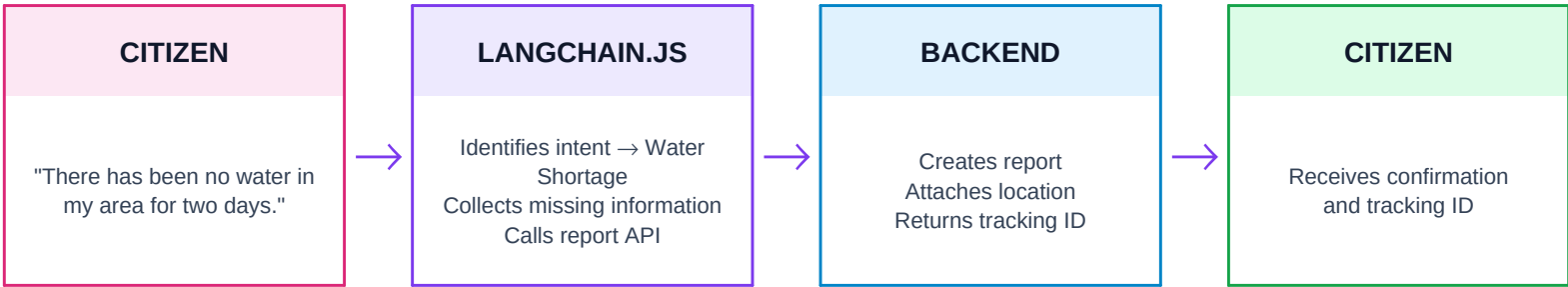
CLIMATE SMART CITY

AI + LangChain.js Role

4. WHERE AI FITS



Example AI Interaction



Key architecture rule: Do not let the LLM decide the actual risk score. The Risk Engine calculates the score; LangChain.js can explain it or interact with the system through controlled tools/APIs.

CLIMATE SMART CITY

Recommended JavaScript / TypeScript Architecture

5. TECHNICAL FLOW

Frontend	React + TypeScript + Vite	Citizen UI + Authority Dashboard
Maps	React-Leaflet + OpenStreetMap	Risk zones + reports + ward map
Backend	Node.js + Express + TypeScript	REST APIs + authentication + workflow
Database	PostgreSQL + PostGIS + Prisma	Reports + locations + climate + risk
Jobs	Redis + BullMQ	Scheduled ingestion + prediction + alerts
AI	LangChain.js + LLM	Assistant + classification + explanations + tool calls
ML	TensorFlow.js (later)	Learned prediction models when sufficient data exists
Storage	Cloudinary / S3	Citizen photos and evidence

Recommended build sequence: Application workflow first → database + maps → risk engine → authority workflow → LangChain.js assistant → ML prediction.